



ECHOCeM: THIRD INTERFACE ECHO SEGMENTATION FOR CEMENT QUALITY ASSESSMENT

DATAChallenge DE DEEP

Auteurs: Víctor Camps Abarca
 Aldric Bahna

Professeurs: Bertrand Michel
 Didier Lime

NANTES, 16 JANVIER 2026

Table des matières

1	Introduction	1
1.1	Contexte	1
1.2	Outils utilisés	1
1.3	Métrique de performance	1
2	Exploration des données	2
2.1	Format des données	2
2.2	Distribution et déséquilibre des données	2
2.3	Pipeline de prétraitement et justification des techniques	3
3	Construction du modèle de référence	4
3.1	Motivations	4
3.2	Architecture générale	4
3.2.1	Structure du projet	4
3.2.2	Description des modules	6
3.2.3	Modèle UNet Baseline	6
3.2.4	Modifications ultérieures	7
4	Premier modèle	8
5	Conclusion	8

Table des figures

1	Distribution des classes dans le dataset	2
2	Pipeline complet de prétraitement : Original $\rightarrow$ Redimensionné $\rightarrow$ Normalisé . .	4
3	Pipeline complet : entraînement (gauche) et inférence (droite)	5

Liste des tableaux

1	Dimensions et nombre de filtres à chaque étape du UNet baseline	7
---	---	---

1 Introduction

1.1 Contexte

Les données étudiées dans ce travail sont constituées d'images ultrasonores acquises lors d'inspections de puits, représentant des coupes radiales du tubage, du ciment et de la formation environnante. Ces images, caractérisées par une forte variabilité spatiale et un niveau de bruit significatif, contiennent une information indirecte mais essentielle sur l'état du ciment, dont la qualité conditionne l'intégrité mécanique et la sécurité du puits. L'enjeu est d'exploiter ces données pour identifier automatiquement, au niveau du pixel, des interfaces physiques clés, en particulier la troisième interface d'écho entre le ciment et la formation géologique. Le problème est formulé comme une tâche de segmentation supervisée d'images, dans laquelle un modèle d'apprentissage automatique doit apprendre à associer à chaque pixel une classe correspondant à une structure physique donnée. L'objectif de ce travail est ainsi de concevoir un pipeline de détection des segmentations, robuste et généralisable, évaluée à l'aide de métriques standard de segmentation (Intersection Over Union IOU), afin de fournir une estimation fiable et automatisée de la qualité du ciment à partir des données ultrasonores.

1.2 Outils utilisés

Nous avons assuré le suivi de version et la collaboration via l'outil GitHub. Le dépôt complet du projet est accessible à l'adresse suivante : [victorcamps70/deep-datachallenge](https://github.com/victorcamps70/deep-datachallenge).

L'implémentation a été réalisée en langage Python, en adoptant une architecture logicielle modulaire. Cette décomposition structurelle a été conçue pour garantir la reproductibilité des expériences et faciliter l'analyse comparative des différentes architectures de modèles. Bien que le projet ait été développé comme un dépôt logiciel standard, le format de rendu imposé a nécessité une adaptation : la structure du projet est détaillée dans le corps du document, et l'intégralité des scripts sources a été reportée en annexe du notebook pour en assurer l'autonomie d'exécution.

La rédaction de ce rapport a été effectuée sous $\text{\LaTeX}$, avec un suivi de version intégré au dépôt du code. Par ailleurs, les soumissions sur la plateforme du challenge sont identifiées sous le compte *Victor.camps2026*.

Dans un souci de transparence méthodologique, nous mentionnons l'usage d'assistants d'intelligence artificielle (modèles Claude Haiku et Gemini-3). Ces outils ont été sollicités principalement pour l'assistance au débogage, la standardisation de la documentation technique (docstrings) et l'aide à la révision stylistique.

1.3 Métrique de performance

La métrique utilisée pour évaluer les modèles dans ce Datachallenge est l'**Intersection Over Union** (IoU). L'IoU est définie mathématiquement comme :

$$\text{IoU} = \frac{\#X \cap Y}{\#X \cup Y}$$

où X est l'ensemble des pixels prédits avec une classe donnée et Y l'ensemble des pixels avec cette classe dans la Ground Truth.

2 Exploration des données

2.1 Format des données

Le dataset d'entraînement contient **4410 patches d'images** provenant de 4 puits distincts (puits 2, 3, 4, 5 et 6). Chaque image est représentée sous forme vectorisée aplatie contenant **43520 pixels**, correspondant à des dimensions originales de 160×272 pixels. Les données d'étiquetage sont des classes de segmentation sémantique multi-classe : **Background** (classe 0), **Casing** (classe 1) et **TIE** (classe 2).

Cependant, le dataset présente une **hétérogénéité dimensionnelle** : certaines images ont des dimensions de 160×160 pixels tandis que d'autres sont en 160×272 pixels. Pour harmoniser le traitement et s'adapter aux exigences des architectures de réseau de neurones convolutifs, une décision de **normalisation spatiale à 160×160 pixels** a été prise. Le redimensionnement est effectué via la bibliothèque **OpenCV** avec interpolation linéaire **INTER_LINEAR**, offrant un excellent compromis entre qualité et performance computationnelle, et améliorant la précision offerte par l'approche du benchmark.

2.2 Distribution et déséquilibre des données

L'analyse du dataset révèle un **déséquilibre significatif** selon deux dimensions critiques pour l'entraînement des modèles.

Déséquilibre par puits : La répartition des patches n'est pas uniforme entre les puits. Les puits 2 et 6 contiennent respectivement environ 1600 et 1500 patches, tandis que le puits 4 en contient environ 600, et le puits 5 seulement 300. Cela représente un **ratio déséquilibré de 1 :5** entre le puits le moins et le plus représenté. Ce déséquilibre peut introduire un biais dans l'entraînement, où les caractéristiques géologiques spécifiques aux puits surreprésentés seraient surpondérées au détriment des autres.

Déséquilibre par classe : Le problème est encore plus prononcé au niveau des classes de segmentation. L'analyse des pixels révèle une distribution fortement imbriquée : la classe **Background** représente environ **85%** des pixels totaux, tandis que **Casing** en représente **9%** et **TIE** seulement **6%**.

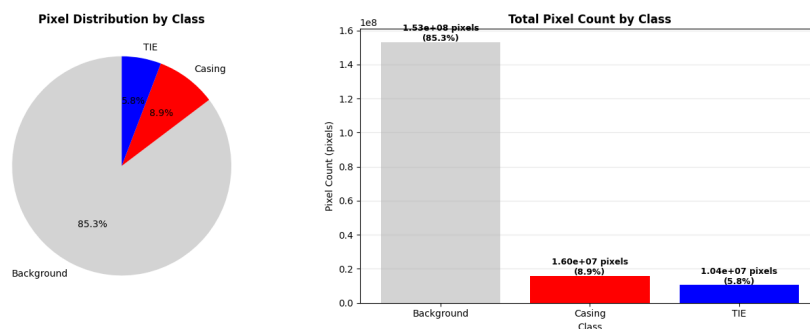


FIGURE 1 – Distribution des classes dans le dataset

Ce déséquilibre extrême (ratio d'environ 15 :1 entre les classes majoritaires et minoritaires) pose un défi majeur pour l'apprentissage supervisé : un modèle naïf pourrait atteindre 85% de

précision en prédisant Background sur tous les pixels, sans capturer les structures d'intérêt.

Ces observations justifient l'implémentation de **techniques de correction du déséquilibre** : l'application de **poids de classe** (class weights) pour pénaliser davantage les erreurs sur les classes minoritaires, l'utilisation de **data augmentation** pour augmenter virtuellement la diversité des données, et l'adoption de fonctions de perte spécialisées comme **Focal Loss** qui amplifient l'apprentissage sur les échantillons difficiles.

2.3 Pipeline de prétraitement et justification des techniques

Le pipeline de prétraitement implémenté suit quatre étapes essentielles pour préparer les données brutes à l'entraînement des réseaux de neurones.

Redimensionnement spatial : Toutes les images sont redimensionnées à la dimension uniforme de 160×160 pixels via **OpenCV** avec interpolation linéaire. Cette étape est cruciale car les architectures CNN exigent des dimensions d'entrée fixes et les images carrées offrent une symétrie géométrique bénéfique pour l'extraction de features.

Le choix de la méthode d'interpolation est **critique et différencié selon le type de données**. Pour les **images d'intensité**, on utilise l'interpolation **INTER_LINEAR** (bilinéaire), qui interpole les valeurs intermédiaires entre pixels pour créer des transitions lisses. Cette approche préserve les détails fins du contenu informatif et produit une image de meilleure qualité pour l'apprentissage du réseau. En revanche, pour les **masques de segmentation** contenant des classes discrètes (0, 1, 2), on utilise l'interpolation **INTER_NEAREST** (plus proche voisin), qui garantit que chaque pixel redimensionné reste une classe valide. Utiliser **INTER_LINEAR** sur les masques créerait des valeurs fractionnaires invalides (0.5, 1.3, etc.), corrompant les étiquettes. Cette distinction méthodologique assure une cohérence sémantique : les images conservent leurs variations d'intensité lisses tandis que les labels de classe demeurent **entiers et valides** après redimensionnement.

Gestion des valeurs manquantes : Lors du redimensionnement, certaines opérations d'interpolation peuvent générer des valeurs NaN. Ces valeurs sont détectées et remplacées par 0 (correspondant à des intensités nulles) pour assurer l'intégrité des données sans perturber significativement le contenu informatif des images.

Normalisation min-max : Les intensités de pixels, initialement dans leur plage brute, sont normalisées à l'intervalle $[0, 1]$ selon la formule $\tilde{x} = \frac{x - x_{min}}{x_{max} - x_{min}}$. Cette normalisation est **critique pour la convergence numérique** des réseaux de neurones : elle assure que les gradients restent dans des plages numériquement stables et que l'initialisation des poids est appropriée aux magnitudes d'entrée.

Voici un exemple illustré :

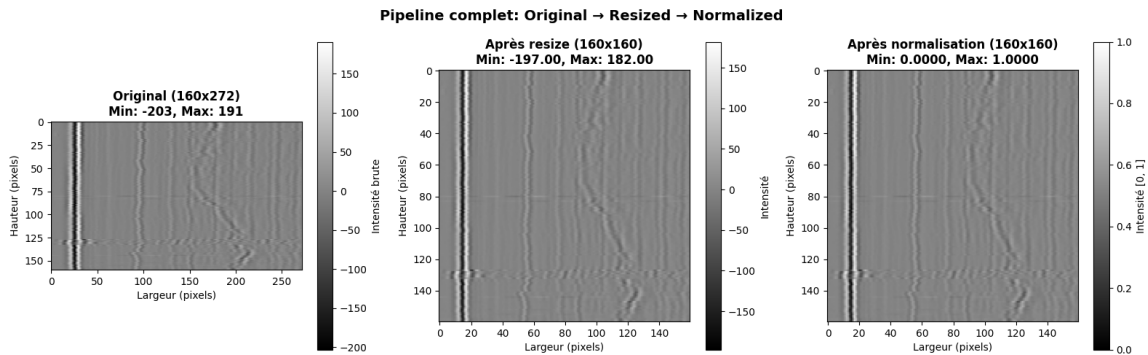


FIGURE 2 – Pipeline complet de prétraitement : Original → Redimensionné → Normalisé

3 Construction du modèle de référence

3.1 Motivations

Nous avons choisi d'utiliser un modèle Unet, c'est-à-dire, un encodeur-décodeur à convolution, développé pour la segmentation d'images médicales. Nous avons eu l'idée en regardant cette vidéo youtube Deep learning pour la segmentation au cours du temps des images en lumière transmise puis nous avons fait des recherches pour valider que ce modèle soit efficace dans ce genre de situations.

Pour les modèles suivants : attention-UNet, pretrained Unet, nous nous sommes basés sur cet article Attention Res-UNet with Guided Decoder for semantic segmentation of brain tumors

3.2 Architecture générale

3.2.1 Structure du projet

Le projet adopte une architecture modulaire organisée en couches fonctionnelles distinctes, chacune responsable d'une étape du pipeline de segmentation. Cette organisation garantit la maintenabilité et permet de substituer facilement des composants sans affecter le reste du système.

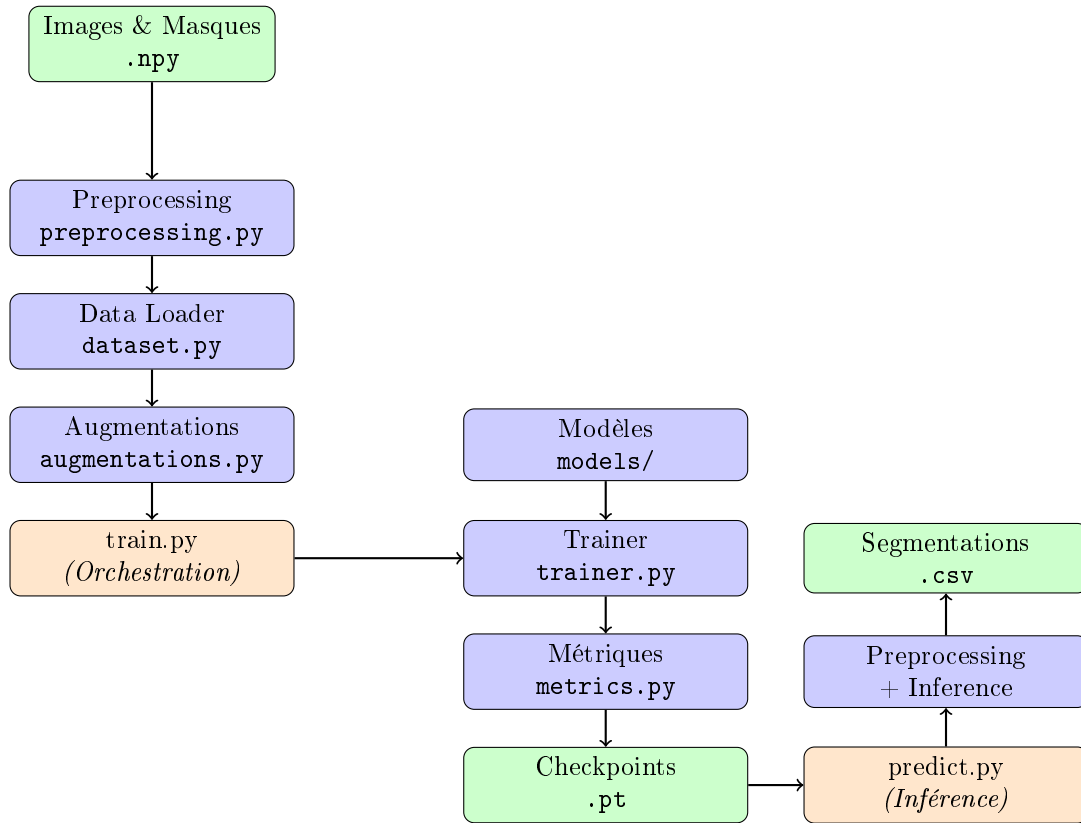


FIGURE 3 – Pipeline complet : entraînement (gauche) et inférence (droite)

Le schéma Figure 3 illustre l'architecture modulaire du projet en deux phases distinctes :

Phase d'entraînement (flux vertical, gauche) : Les données brutes (images ultrasonores et masques en format `.npz`) sont d'abord prétraitées via normalisation et interpolation. Elles sont ensuite chargées dans des DataLoaders PyTorch avec splits train/validation stratifiés par puits pour éviter la fuite de données. Pendant l'entraînement, chaque batch subit des augmentations aléatoires (rotations, flips, bruit) pour améliorer la robustesse. Le script **train.py** orchestre l'ensemble du processus, initialisant le modèle, l'optimiseur et les hyperparamètres. La boucle d'entraînement dans **trainer.py** effectue à chaque epoch un forward pass, calcule la perte pondérée par classe, rétropropage les gradients, et met à jour les poids du réseau. Les métriques d'évaluation (IoU par classe) sont calculées en fin de chaque epoch sur l'ensemble de validation. Le meilleur modèle est automatiquement sauvegardé en checkpoint `.pt` via Early Stopping dès que la métrique de validation s'améliore.

Phase d'inférence (flux horizontal, droite) : Une fois l'entraînement terminé, le script **predict.py** charge le meilleur checkpoint et l'utilise pour générer des prédictions segmentation sur des images test. Le preprocessing est réappliqué aux nouvelles images pour garantir la cohérence. Les segmentations prédites (cartes de probabilité par classe) sont ensuite formatées et exportées en fichiers `.csv` pour l'évaluation externe.

Cette séparation entre les deux phases assure une distinction claire entre la phase de

développement et la phase de déploiement, facilitant ainsi la maintenance et l'utilisation du modèle en production.

3.2.2 Description des modules

- **preprocessing.py** : Normalisation min-max, interpolation des images manquantes, conversion des masques en format compatible PyTorch.
- **dataset.py** : Chargement des données, création des DataLoaders PyTorch, gestion des splits train/validation avec stratification par puits.
- **augmentations.py** : Implémentation des transformations géométriques (rotations, flips) et du bruit Gaussien appliqués dynamiquement en entraînement.
- **models/** : Définitions des architectures UNet, UNet avec encodeur pré-entraîné, et variantes avec attention mechanisms.
- **trainer.py** : Boucle d'entraînement complète incluant forward pass, calcul de loss, backward propagation, et sauvegarde du meilleur modèle via Early Stopping.
- **metrics.py** : Calcul de l'Intersection over Union (IoU) pour chaque classe et accumulation des métriques moyennes via la classe **MetricsTracker**. Assure le suivi continu des performances pendant l'entraînement et la validation.
- **train.py** : Point d'entrée principal orchestrant l'entraînement avec arguments en ligne de commande pour configurer le modèle, le dataset et les hyperparamètres.
- **predict.py** : Interface d'inférence permettant de générer des prédictions segmentation sur de nouvelles images.

3.2.3 Modèle UNet Baseline

Le modèle UNet *baseline* constitue la référence de ce travail. Son architecture suit la structure encodeur-décodeur classique proposée par [?], adaptée à notre contexte d'images ultrasonores monocanal (160×160 pixels).

Architecture générale :

- **Encodeur** : 4 niveaux de convolution descendante avec max pooling
- **Goulot d'étranglement (Bottleneck)** : Bloc de convolution au niveau le plus profond
- **Décodeur** : 4 niveaux de déconvolution ascendante avec skip connections
- **Skip connections** : Concaténation des activations de l'encodeur aux niveaux correspondants du décodeur, préservant l'information spatiale

Blocs constitutifs :

Chaque niveau de l'encodeur et du décodeur utilise un **ConvBlock** composé de :

$$\boxed{\text{Conv2d}(k = 3) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU}} \quad (\text{appliqué 2 fois})$$

Cette structure double-convolution avec normalisation par batch stabilise l'apprentissage et améliore la convergence. Le padding de 1 pixel préserve les dimensions spatiales au sein de chaque bloc.

Dimensions au fil du réseau :

Étape	Entrée	Sortie	Filtres
Input	-	$1 \times 160 \times 160$	-
Encoder-1	160^2	$64 \times 80 \times 80$	64
Encoder-2	80^2	$128 \times 40 \times 40$	128
Encoder-3	40^2	$256 \times 20 \times 20$	256
Encoder-4	20^2	$512 \times 10 \times 10$	512
Bottleneck	10^2	$1024 \times 10 \times 10$	1024
Decoder-1	10^2	$512 \times 20 \times 20$	512
Decoder-2	20^2	$256 \times 40 \times 40$	256
Decoder-3	40^2	$128 \times 80 \times 80$	128
Decoder-4	80^2	$64 \times 160 \times 160$	64
Output	160^2	$3 \times 160 \times 160$	3 classes

TABLE 1 – Dimensions et nombre de filtres à chaque étape du UNet baseline

Ce modèle baseline sert de point de comparaison pour les architectures ultérieures (avec Dropout, avec Focal Loss, avec transfer learning).

3.2.4 Modifications ultérieures

Ajout de Dropout : Nous avons expérimenté l'ajout de couches de dropout afin de réduire le risque de surapprentissage.

Dans cette variante, nous avons intégré du dropout (avec probabilité $p = 0.3$) après chaque activation ReLU du **ConvBlock**. La motivation sous-jacente reposait sur plusieurs considérations théoriques :

- **Prévention de la co-adaptation** : Le dropout empêche les neurones d'apprendre des dépendances co-adaptées, ce qui peut être particulièrement utile lorsque le dataset est petit (4410 patches seulement).
- **Amélioration de la généralisation** : En encourageant le réseau à apprendre des features redondantes et interprétables, le dropout améliore généralement la capacité de généralisation sur des données non vues.
- **Réduction du surapprentissage** : Avec un dataset réduit et limité à 12 puits, le réseau risque naturellement de mémoriser les patterns locaux plutôt que d'apprendre des features généralisables. Le dropout ajoute du bruit stochastique qui agit comme un régulariseur implicite.

Bien que cette approche soit théoriquement solide, les résultats empiriques ont montré que l'ajout de dropout au modèle baseline a **dégradé les performances**. Contrairement aux attentes, la désactivation aléatoire de neurones n'a pas conduit à une meilleure généralisation, mais a au contraire perturbé la capacité du modèle à apprendre des patterns cohérents. Cela peut s'expliquer par plusieurs facteurs : d'une part, le dataset, bien que limité, possède une structure suffisamment cohérente (provenant des mêmes 12 puits) pour que la couche de dropout introduise une perturbation trop agressive ; d'autre part, d'autres formes de régularisation (comme la data augmentation et les poids de classe) suffisaient déjà à maîtriser l'overfitting sans nécessiter de bruit supplémentaire. Cette observation souligne que certaines techniques de régularisation peuvent être contre-productives lorsqu'appliquées naïvement, et qu'une calibration fine est nécessaire en fonction de la complexité du modèle et des caractéristiques du dataset.

Ajout de bruit Gaussien dans l'augmentation : Parallèlement, nous avons enrichi le pipeline d'augmentation en intégrant du bruit Gaussien appliqué dynamiquement pendant l'entraînement. Cette augmentation consiste à ajouter un bruit aléatoire (écart-type $\sigma = 0.02$) à chaque image du batch avec une probabilité de 30%, en plus des transformations géométriques existantes (rotations, flips).

La motivation pour cette modification reposait sur plusieurs observations pratiques et théoriques :

- **Réalisme des données ultrasonores :** Les images ultrasonores acquises in situ contiennent naturellement du bruit d'acquisition dû aux appareils de détection, aux artefacts de propagation, et aux variations du milieu. En exposant le modèle à du bruit synthétique durant l'entraînement, nous le préparons à généraliser sur des images réelles bruyantes.
- **Augmentation sans distorsion sémantique :** Contrairement aux augmentations agressives (déformations élastiques, flou), le bruit Gaussien préserve la structure et les contours des classes de segmentation. Il augmente la variabilité sans détruire l'information pertinente.
- **Robustesse accrue :** Un modèle entraîné sur des données bruyantes devient moins sensible aux petites variations d'acquisition, améliorant potentiellement sa robustesse face à des conditions d'imagerie variables.

Les résultats de cette modification ont révélé un aspect intéressant : l'efficacité du bruit Gaussien dépend fortement de l'architecture utilisée. Lorsqu'appliqué au modèle baseline seul, l'augmentation par bruit a paradoxalement **empiré les performances**. Cette dégradation peut s'expliquer par le fait que le baseline, sans mécanismes sophistiqués pour filtrer ou apprendre les features pertinentes, a du mal à extraire du signal utile lorsque le bruit synthétique introduit une variabilité excessive.

Cependant, lorsque le bruit Gaussien a été appliqué à d'autres architectures du projet (notamment les variantes avec transfer learning et encodeurs pré-entraînés), l'augmentation s'est avérée **bénéfique**. Les modèles pré-entraînés, disposant déjà de features générales robustes apprises sur ImageNet, ont mieux su exploiter le bruit comme régularisateur et ont profité de la robustesse accrue qu'il confère.

4 Premier modèle

5 Conclusion